Assignment-16.3

SHIVANI PAMBIDI
BATCH 41
2303A52172

Task 1 – Hospital Management Database

PROMPT:

Create normalized schema for Hospital Management System Tables: Doctors, Patients, Appointments

Include primary keys, foreign keys, constraints, valid date checks

Code :

sitantcoding > hospital_management_schema.sql

```sql
-- Hospital Management System (sqlite)
-- Normalized schema: Doctors, Patients, Appointments

BEGIN;

CREATE TABLE doctors (
    doctor_id BIGSERIAL PRIMARY KEY,    "BIGSERIAL": Unknown word.
    first_name VARCHAR(60) NOT NULL,    "VARCHAR": Unknown word.
    last_name VARCHAR(60) NOT NULL,    "VARCHAR": Unknown word.
    specialization VARCHAR(100) NOT NULL,    "VARCHAR": Unknown word.
    email VARCHAR(120) NOT NULL UNIQUE,    "VARCHAR": Unknown word.
    phone VARCHAR(20) NOT NULL UNIQUE,    "VARCHAR": Unknown word.
    license_number VARCHAR(40) NOT NULL UNIQUE,    "VARCHAR": Unknown word.
    years_experience INT NOT NULL DEFAULT 0,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_doctors_names_not_blank
        CHECK (BTRIM(first_name) <> '' AND BTRIM(last_name) <> ''),    "BTRIM": Unknown word.
    CONSTRAINT chk_doctors_specialization_not_blank
        CHECK (BTRIM(specialization) <> ''),    "BTRIM": Unknown word.
    CONSTRAINT chk_doctors_experience_non_negative
```

OBLEMS 23 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 23 SONARQUBE

```sql
CONSTRAINT chk_doctors_experience_non_negative
CHECK (years_experience >= 0)
);

CREATE TABLE patients (
patient_id BIGSERIAL PRIMARY KEY,
```

```
first_name VARCHAR(60) NOT NULL,
last_name VARCHAR(60) NOT NULL,
date_of_birth DATE NOT NULL,
gender VARCHAR(20) NOT NULL,
email VARCHAR(120) UNIQUE,
phone VARCHAR(20) NOT NULL UNIQUE,
blood_group VARCHAR(5),
emergency_contact_name VARCHAR(120),
emergency_contact_phone VARCHAR(20),
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT chk_patients_names_not_blank
CHECK (BTRIM(first_name) <> '' AND BTRIM(last_name) <> ''),
CONSTRAINT chk_patients_valid_dob
CHECK (date_of_birth <= CURRENT_DATE),
CONSTRAINT chk_patients_gender
CHECK (gender IN ('Male', 'Female', 'Other')),
CONSTRAINT chk_patients_blood_group
CHECK (
blood_group IS NULL
OR blood_group IN ('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-')
)
);
```

Queries and output:

Query 1:

SELECT * FROM Appointments WHERE doctor_id = 1; Query 2

SELECT * FROM Appointments WHERE patient_id = 1; Query 3:

SELECT doctor_id, COUNT(DISTINCT patient_id) FROM Appointments GROUP BY doctor_id

Output:

```
============================================================
Task 1 - Hospital Management Database
============================================================
Output 1: List all appointments for a specific doctor
Columns: appointment_id | appointment_ts | status | patient_name
1 | 2026-03-16 16:44:53 | completed | Rohit Kumar

Output 2: Retrieve patient history by patient ID
Columns: patient_name | appointment_ts | doctor_name | reason | diagnosis_notes
Rohit Kumar | 2026-03-16 16:44:53 | Dr. Asha Reddy | Chest pain | Mild condition

Output 3: Count total patients treated by each doctor
Columns: doctor_name | total_patients_treated
Dr. Vivek Rao | 1
Dr. Asha Reddy | 1
```

Justification:

The Hospital Management Database is designed using three tables: Doctors, Patients, and Appointments. The schema is normalized to avoid data duplication and maintain consistency. Primary keys are used to uniquely identify each record, and foreign keys are used to connect related tables. Constraints like NOT NULL, UNIQUE, and valid dates ensure data accuracy. SQL queries with joins help to easily retrieve doctor appointments, patient history, and count of patients per doctor.

Task 2 – Real-Time Application: Online Shopping Database

Prompt:

Create schema for e-commerce system Tables: Users, Products, Orders, OrderDetails and Include relationships and normalization

Code:

```sql
-- Task 2 - E-Commerce System (SQLite)
-- Normalized schema: users, products, orders, order_details

PRAGMA foreign_keys = ON;

BEGIN TRANSACTION;

CREATE TABLE users (
user_id INTEGER PRIMARY KEY AUTOINCREMENT,
full_name TEXT NOT NULL,
email TEXT NOT NULL UNIQUE,
phone TEXT UNIQUE,
password_hash TEXT NOT NULL,
is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0, 1)),
created_at TEXT NOT NULL DEFAULT (datetime('now')),
updated_at TEXT NOT NULL DEFAULT (datetime('now')),
CHECK (trim(full_name) <> ''),
CHECK (trim(email) <> ''),
CHECK (instr(email, '@') > 1)
);

CREATE TABLE products (
product_id INTEGER PRIMARY KEY AUTOINCREMENT,
product_name TEXT NOT NULL,
sku TEXT NOT NULL UNIQUE,
description TEXT,
unit_price NUMERIC NOT NULL,
stock_quantity INTEGER NOT NULL DEFAULT 0,
is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0, 1)),
created_at TEXT NOT NULL DEFAULT (datetime('now')),
updated_at TEXT NOT NULL DEFAULT (datetime('now')),
CHECK (trim(product_name) <> ''),
CHECK (trim(sku) <> ''),
CHECK (unit_price >= 0),
CHECK (stock_quantity >= 0)
);
```

```sql
CREATE TABLE orders (
order_id INTEGER PRIMARY KEY AUTOINCREMENT,
user_id INTEGER NOT NULL,
order_date TEXT NOT NULL DEFAULT (datetime('now')),
order_status TEXT NOT NULL DEFAULT 'Pending',
payment_status TEXT NOT NULL DEFAULT 'Unpaid',
shipping_address TEXT NOT NULL,
order_total NUMERIC NOT NULL DEFAULT 0,
created_at TEXT NOT NULL DEFAULT (datetime('now')),
updated_at TEXT NOT NULL DEFAULT (datetime('now')),
FOREIGN KEY (user_id)
REFERENCES users (user_id)
ON UPDATE CASCADE
ON DELETE RESTRICT,
CHECK (date(order_date) IS NOT NULL),
CHECK (order_status IN ('Pending', 'Confirmed', 'Shipped', 'Delivered',
'Cancelled')),
CHECK (payment_status IN ('Unpaid', 'Paid', 'Refunded', 'Failed')),
CHECK (trim(shipping_address) <> ''),
CHECK (order_total >= 0)
);

CREATE TABLE order_details (
order_detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
order_id INTEGER NOT NULL,
product_id INTEGER NOT NULL,
quantity INTEGER NOT NULL,
unit_price NUMERIC NOT NULL,
line_total NUMERIC NOT NULL,
created_at TEXT NOT NULL DEFAULT (datetime('now')),
updated_at TEXT NOT NULL DEFAULT (datetime('now')),
FOREIGN KEY (order_id)
REFERENCES orders (order_id)
ON UPDATE CASCADE
ON DELETE CASCADE,
FOREIGN KEY (product_id)
REFERENCES products (product_id)
ON UPDATE CASCADE
ON DELETE RESTRICT,
CHECK (quantity > 0),
```

```sql
CHECK (unit_price >= 0),
CHECK (line_total >= 0),
UNIQUE (order_id, product_id)
);

CREATE INDEX idx_orders_user_id
ON orders (user_id);

CREATE INDEX idx_orders_order_date
ON orders (order_date);

CREATE INDEX idx_order_details_order_id
ON order_details (order_id);

CREATE INDEX idx_order_details_product_id
ON order_details (product_id);

COMMIT;
```

Queries and output

```
============================================================
Task 2 - Online Shopping Database
============================================================
Output 1: Retrieve all orders by a user
Columns: order_id | order_date | order_status | order_total
2 | 2026-03-13 16:44:53 | shipped | 3298.0
1 | 2026-03-08 16:44:53 | paid | 1598.0

Output 2: Find the most purchased product
Columns: product_name | total_units_sold
Wireless Mouse | 3

Output 3: Calculate total revenue in current month
Columns: total_revenue
4896.0
```

Task 3 – Library Database

Prompt:

Create library schema

Tables: Books, Members, Loans Suggest indexes for performance Code

```
- Task - Library Management System (SQLite)
-- Schema + sample data
```

```sql
PRAGMA foreign_keys = ON;

BEGIN TRANSACTION;

CREATE TABLE books (
book_id INTEGER PRIMARY KEY AUTOINCREMENT,
isbn TEXT NOT NULL UNIQUE,
title TEXT NOT NULL,
author_name TEXT NOT NULL,
category TEXT,
published_year INTEGER,
total_copies INTEGER NOT NULL DEFAULT 1,
available_copies INTEGER NOT NULL DEFAULT 1,
shelf_code TEXT,
created_at TEXT NOT NULL DEFAULT (datetime('now')),
updated_at TEXT NOT NULL DEFAULT (datetime('now')),
CHECK (trim(isbn) <> ''),
CHECK   (trim(title)   <>   ''),
CHECK (trim(author_name) <> ''),
CHECK (published_year IS NULL OR published_year BETWEEN 1450 AND 2100),
CHECK (total_copies >= 0),
CHECK (available_copies >= 0),
CHECK (available_copies <= total_copies)
);

CREATE TABLE members (
member_id INTEGER PRIMARY KEY AUTOINCREMENT,
full_name TEXT NOT NULL,
email TEXT NOT NULL UNIQUE,
```

```sql
phone TEXT UNIQUE,
membership_date TEXT NOT NULL DEFAULT (date('now')),
status TEXT NOT NULL DEFAULT 'Active',
created_at TEXT NOT NULL DEFAULT (datetime('now')),
updated_at TEXT NOT NULL DEFAULT (datetime('now')),
CHECK (trim(full_name) <> ''),
CHECK (instr(email, '@') > 1),
CHECK (status IN ('Active', 'Suspended', 'Inactive')),
CHECK (date(membership_date) IS NOT NULL)
);

CREATE TABLE loans (
loan_id INTEGER PRIMARY KEY AUTOINCREMENT,
book_id INTEGER NOT NULL,
member_id INTEGER NOT NULL,
loan_date TEXT NOT NULL,
due_date TEXT NOT NULL,
return_date TEXT,
loan_status TEXT NOT NULL DEFAULT 'Borrowed',
fine_amount NUMERIC NOT NULL DEFAULT 0,
created_at TEXT NOT NULL DEFAULT (datetime('now')),
updated_at TEXT NOT NULL DEFAULT (datetime('now')),
FOREIGN KEY (book_id)
REFERENCES books (book_id)
ON UPDATE CASCADE
ON DELETE RESTRICT,
FOREIGN KEY (member_id)
REFERENCES members (member_id)
ON UPDATE CASCADE
ON DELETE RESTRICT,
CHECK (date(loan_date) IS NOT NULL),
CHECK (date(due_date) IS NOT NULL),
CHECK (return_date IS NULL OR date(return_date) IS NOT NULL),
CHECK (date(due_date) >= date(loan_date)),
CHECK (return_date IS NULL OR date(return_date) >= date(loan_date)),
CHECK (loan_status IN ('Borrowed', 'Returned', 'Overdue', 'Lost')),
CHECK (fine_amount >= 0)
);

-- Suggested performance indexes
```

```sql
CREATE INDEX idx_books_title
ON books (title);

CREATE INDEX idx_books_author
ON books (author_name);

CREATE INDEX idx_members_email
ON members (email);

CREATE INDEX idx_loans_member_status
ON loans (member_id, loan_status);

CREATE INDEX idx_loans_book_status
ON loans (book_id, loan_status);

CREATE INDEX idx_loans_due_date
ON loans (due_date);

-- Sample books
INSERT OR IGNORE INTO books
(isbn, title, author_name, category, published_year, total_copies, available_copies,
shelf_code)
VALUES
('9780131103627', 'The C Programming Language', 'Brian W. Kernighan', 'Programming',
1988, 5, 3, 'CS-A1'),
('9780132350884', 'Clean Code', 'Robert C. Martin', 'Software Engineering', 2008, 4,
2, 'CS-B2'),
('9780262033848', 'Introduction to Algorithms', 'Thomas H. Cormen', 'Algorithms',
2009, 3, 1, 'CS-C3'),
('9780596009205', 'Head First Java', 'Kathy Sierra', 'Programming', 2005, 6, 4, 'CS-
D4');

-- Sample members
INSERT OR IGNORE INTO members
(full_name, email, phone, membership_date, status)
VALUES
('Riya Verma', 'riya.verma@example.com', '9100000001', '2026-01-10', 'Active'),
('Arun Patel', 'arun.patel@example.com', '9100000002', '2026-02-04', 'Active'),
('Sneha Nair', 'sneha.nair@example.com', '9100000003', '2026-02-15', 'Active');
```

```sql
-- Sample loans
INSERT OR IGNORE INTO loans
(book_id, member_id, loan_date, due_date, return_date, loan_status, fine_amount)
VALUES
(1, 1, '2026-03-10', '2026-03-20', NULL, 'Borrowed', 0),
(2, 1, '2026-03-01', '2026-03-10', '2026-03-09', 'Returned', 0),
(3, 2, '2026-03-05', '2026-03-15', NULL, 'Borrowed', 0),
(4, 3, '2026-02-20', '2026-03-01', '2026-03-03', 'Returned', 20);

COMMIT;
```

Queries and output:

```
==============================================================
Task 3 - Library Database
==============================================================
Output 1: Retrieve all books currently issued
Columns: title | issued_to | loan_date | due_date
DBMS Fundamentals | Rahul Singh | 2026-02-11 | 2026-03-13
AI Basics | Ananya Das | 2026-03-08 | 2026-03-28

Output 2: Find overdue books (loan date > 30 days)
Columns: title | member_name | loan_date | days_since_loan
DBMS Fundamentals | Rahul Singh | 2026-02-11 | 35

Output 3: Count number of books loaned by each member
Columns: member_name | total_books_loaned
Rahul Singh | 1
Ananya Das | 1
```

Justification:

Copilot Index improves search speed
suggested indexing on date column

Task 4: Optimize Queries

Prompt : ptimize query using JOIN instead of subquery

Code:

-- Original

SELECT * FROM Books WHERE author_id IN
(SELECT author_id FROM Authors WHERE country='UK');

-- Optimized SELECT b.*

FROM Books b

JOIN Authors a ON b.author_id = a.author_id WHERE
a.country='UK';

```
-- Original
SELECT * FROM Books WHERE author_id IN
(SELECT author_id FROM Authors WHERE country='UK');

-- Optimized
```

```sql
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id WHERE
a.country='UK';
```

Output:

```
============================================================
Task 4 - Optimize Queries
============================================================
Original query rows: 1
Optimized query rows: 1
Result equivalence: YES
Conclusion: Both queries return same result. Optimized query is preferred for scalability.
```

Task 5: University Course Registration

Prompt:

ceate schema for Students, Courses, Faculty, Registrations Define relationships and constraints

Code:

```sql
- Task 5 - University Course Registration (SQLite)
```

```sql
-- Schema + sample data + required queries

PRAGMA foreign_keys = ON;

BEGIN TRANSACTION;

CREATE TABLE faculty (
faculty_id INTEGER PRIMARY KEY AUTOINCREMENT,
full_name TEXT NOT NULL,
email TEXT NOT NULL UNIQUE,
department TEXT NOT NULL,
designation TEXT,
hire_date TEXT,
is_active INTEGER NOT NULL DEFAULT 1 CHECK (is_active IN (0, 1)),
created_at TEXT NOT NULL DEFAULT (datetime('now')),
updated_at TEXT NOT NULL DEFAULT (datetime('now')),
CHECK (trim(full_name) <> ''),
CHECK (instr(email, '@') > 1),
CHECK (trim(department) <> ''),
CHECK (hire_date IS NULL OR date(hire_date) IS NOT NULL)
);

CREATE TABLE students (
student_id INTEGER PRIMARY KEY AUTOINCREMENT,
roll_no TEXT NOT NULL UNIQUE,
full_name TEXT NOT NULL,
email TEXT NOT NULL UNIQUE,
phone TEXT UNIQUE,
program TEXT NOT NULL,
semester INTEGER NOT NULL,
enrollment_date TEXT NOT NULL DEFAULT (date('now')),
status TEXT NOT NULL DEFAULT 'Active',
created_at TEXT NOT NULL DEFAULT (datetime('now')),
updated_at TEXT NOT NULL DEFAULT (datetime('now')),
CHECK (trim(roll_no) <> ''),
CHECK (trim(full_name) <> ''),
CHECK (instr(email, '@') > 1),
CHECK (trim(program) <> ''),
CHECK (semester BETWEEN 1 AND 12),
CHECK (date(enrollment_date) IS NOT NULL),
```

```sql
CHECK (status IN ('Active', 'Graduated', 'Dropped'))
);

CREATE TABLE courses (
course_id INTEGER PRIMARY KEY AUTOINCREMENT,
course_code TEXT NOT NULL UNIQUE,
course_title TEXT NOT NULL,
credits INTEGER NOT NULL,
department TEXT NOT NULL,
faculty_id INTEGER NOT NULL,
max_capacity INTEGER NOT NULL DEFAULT 60,
created_at TEXT NOT NULL DEFAULT (datetime('now')),
updated_at TEXT NOT NULL DEFAULT (datetime('now')),
FOREIGN KEY (faculty_id)
REFERENCES faculty (faculty_id)
ON UPDATE CASCADE
ON DELETE RESTRICT,
CHECK (trim(course_code) <> ''),
CHECK (trim(course_title) <> ''),
CHECK (trim(department) <> ''),
CHECK (credits BETWEEN 1 AND 6),
CHECK (max_capacity > 0)
);

CREATE TABLE registrations (
registration_id INTEGER PRIMARY KEY AUTOINCREMENT,
student_id INTEGER NOT NULL,
course_id INTEGER NOT NULL,
academic_year TEXT NOT NULL,
semester_term TEXT NOT NULL,
registration_date TEXT NOT NULL DEFAULT (date('now')),
grade TEXT,
status TEXT NOT NULL DEFAULT 'Enrolled',
created_at TEXT NOT NULL DEFAULT (datetime('now')),
updated_at TEXT NOT NULL DEFAULT (datetime('now')),
FOREIGN KEY (student_id)
REFERENCES students (student_id)
ON UPDATE CASCADE
ON DELETE CASCADE,
FOREIGN KEY (course_id)
```

```sql
REFERENCES courses (course_id)
ON UPDATE CASCADE
ON DELETE CASCADE,
CHECK (academic_year GLOB '[0-9][0-9][0-9][0-9]-[0-9][0-9][0-9][0-9]'),
CHECK (semester_term IN ('Odd', 'Even', 'Summer')),
CHECK (date(registration_date) IS NOT NULL),
CHECK (grade IS NULL OR grade IN ('O', 'A+', 'A', 'B+', 'B', 'C', 'P', 'F')),
CHECK (status IN ('Enrolled', 'Completed', 'Dropped', 'Withdrawn')),
UNIQUE (student_id, course_id, academic_year, semester_term)
);

-- Suggested performance indexes
CREATE INDEX idx_students_program_semester
ON students (program, semester);

CREATE INDEX idx_courses_faculty
ON courses (faculty_id);

CREATE INDEX idx_registrations_course
ON registrations (course_id);

CREATE INDEX idx_registrations_student
ON registrations (student_id);

CREATE INDEX idx_registrations_course_status
ON registrations (course_id, status);

-- Sample faculty
INSERT OR IGNORE INTO faculty (full_name, email, department, designation, hire_date,
is_active)
VALUES
('Dr. Anil Kumar', 'anil.kumar@sru.edu', 'Computer Science', 'Professor', '2015-06-
10', 1),
('Dr. Priya Menon', 'priya.menon@sru.edu', 'Computer Science', 'Associate Professor',
'2018-01-12', 1),
('Dr. Rakesh Iyer', 'rakesh.iyer@sru.edu', 'Electronics', 'Assistant Professor',
'2020-08-01', 1);

-- Sample students
```

```sql
INSERT OR IGNORE INTO students (roll_no, full_name, email, phone, program, semester,
enrollment_date, status)
VALUES
('SRU23CSE001', 'Aman Verma', 'aman.verma@sru.edu', '9800000001', 'B.Tech CSE', 5,
'2023-08-01', 'Active'),
('SRU23CSE002', 'Nisha Rao', 'nisha.rao@sru.edu', '9800000002', 'B.Tech CSE', 5,
'2023-08-01', 'Active'),
('SRU22ECE010', 'Rahul Nair', 'rahul.nair@sru.edu', '9800000010', 'B.Tech ECE', 7,
'2022-08-01', 'Active'),
('SRU24CSE005', 'Megha Shah', 'megha.shah@sru.edu', '9800000005', 'B.Tech CSE', 3,
'2024-08-01', 'Active');

-- Sample courses
INSERT OR IGNORE INTO courses (course_code, course_title, credits, department,
faculty_id, max_capacity)
VALUES
('CS301', 'Database Management Systems', 4, 'Computer Science', 1, 70),
('CS302', 'Operating Systems', 4, 'Computer Science', 1, 65),
('CS303', 'Design and Analysis of Algorithms', 4, 'Computer Science', 1, 60),
('CS304', 'Computer Networks', 3, 'Computer Science', 2, 60),
('EC401', 'Digital Signal Processing', 4, 'Electronics', 3, 55);

-- Sample registrations
INSERT OR IGNORE INTO registrations (student_id, course_id, academic_year,
semester_term, registration_date, grade, status)
VALUES
(1, 1, '2025-2026', 'Even', '2026-01-08', NULL, 'Enrolled'),
(1, 2, '2025-2026', 'Even', '2026-01-08', NULL, 'Enrolled'),
(1, 4, '2025-2026', 'Even', '2026-01-08', NULL, 'Enrolled'),
(2, 1, '2025-2026', 'Even', '2026-01-09', NULL, 'Enrolled'),
(2, 3, '2025-2026', 'Even', '2026-01-09', NULL, 'Enrolled'),
(3, 5, '2025-2026', 'Even', '2026-01-10', NULL, 'Enrolled'),
(4, 1, '2025-2026', 'Even', '2026-01-10', NULL, 'Enrolled'),
(4, 4, '2025-2026', 'Even', '2026-01-10', NULL, 'Enrolled');

COMMIT;
```

Query and output:

```
============================================================
Task 5 - University Course Registration
============================================================
Output 1: List all students enrolled in a specific course (CS301)
Columns: course_code | course_title | roll_no | student_name
CS301 | Database Systems | SRU-001 | Nikhil Reddy
CS301 | Database Systems | SRU-002 | Priya Jain

Output 2: Find faculty members teaching more than 2 courses
Columns: faculty_name | total_courses
Prof. S. Ramesh | 3

Output 3: Retrieve courses with highest number of registrations
Columns: course_code | course_title | registrations
CS301 | Database Systems | 2
```

Justification:

Copilot ensured many-to-many mapping
Queries optimized with JOIN + GROUP BY